Guía para entender y explicar el proyecto

Esta guía explica, en lenguaje simple, qué hace el programa y cómo responder preguntas durante la demostración.

1. La idea en una frase

Es un **motor 3D hecho a mano**: calcula en el CPU dónde cae cada triángulo en la pantalla, decide el color de cada píxel y guarda el resultado en un arreglo de píxeles (el *framebuffer*). Raylib solo abre la ventana, lee el teclado y muestra ese arreglo como una imagen. **Todo lo 3D lo hace nuestro código.**

2. Qué pasa en cada cuadro (`main.rs`, bucle `while`)

1. **Leer teclas** y calcular `delta_time` (segundos desde el cuadro anterior).
2. **Mover la nave**: las flechas giran la vista; WASD/Espacio/Ctrl suman empuje a la velocidad; el rozamiento la frena.
3. **Gravedad, control local y colisión** (ver sección 5).
4. **Colocar la cámara**: siempre detrás y arriba de la nave, calculada desde su posición. No tiene vida propia.
5. **Limpiar** pantalla y profundidad, **dibujar estrellas**.
6. **Dibujar** el Sol, los planetas (con sus modelos de superficie) y la nave.
7. **Mostrar** el framebuffer con el HUD de texto.

Todo movimiento se multiplica por `delta_time`, por eso la velocidad no depende de los FPS.

3. El pipeline gráfico (lo más preguntado)

Cada vértice de un modelo pasa por estas etapas (`shaders.rs → vertex_shader`, matrices en `matrix.rs`):

Etapa	Qué hace	Matriz
Modelo	Coloca el objeto en el mundo: escala, rota y traslada	`create_model_matrix`
Vista	Mueve el mundo para que la cámara quede en el origen mirando adelante	`create_view_matrix` (lookAt)
Proyección	Perspectiva: lo lejano se ve más chico	`create_projection_matrix`
División por w	Pasa a coordenadas normalizadas (NDC)	—
Viewport	Convierte NDC a píxeles de la ventana	`create_viewport_matrix`

Luego, **por cada triángulo** (`triangle.rs`):

- Se calcula su caja en pantalla y se recorre cada píxel de esa caja.
- Con **coordenadas baricéntricas** se sabe si el píxel está dentro del triángulo y cuánto pesa cada vértice. Con esos pesos se **interpolan** profundidad, normal y posición.
- Se calcula la **luz difusa (Lambert)**: producto punto entre la normal y la dirección hacia el Sol.

- Cada píxel pasa por el **fragment shader** (`fragment_shader`), que decide el color final.
- El **depth buffer** (`framebuffer.rs → point`) solo deja pintar el píxel si está más cerca que lo que ya había. Así los objetos cercanos tapan a los lejanos.

Un detalle importante: en raylib-rs, `A * B` entre matrices equivale a B·A. Por eso en `create_model_matrix` el producto está escrito al revés (`scale * rotation * translation`): así se aplica primero la escala, luego la rotación y al final la traslación.

4. Cómo se generan los planetas (`planet_mesh.rs`)

1. Se parte de un **icosaedro** (20 triángulos).
2. Se **subdivide** recursivamente: cada triángulo se parte en 4 con puntos medios y se proyectan a la esfera. Con 2, 3 y 4 subdivisiones salen **320, 1280 y 5120 triángulos** (son los tres niveles de detalle).
3. Se **deforma** cada vértice empujándolo hacia afuera o adentro según **ruido fractal OpenSimplex** (`TerrainGen::height`). Eso crea montañas y valles.
4. Se calculan las **normales por cara** (aspecto low-poly, tipo Astroneer).

Todo esto ocurre **una sola vez al arrancar**, antes de rasterizar.

5. Navegación

- **Aceleración y frenado progresivos**: la velocidad cambia poco a poco (`THRUST_ACCEL`, `DRAG_*`). El giro también se acerca de forma gradual a la velocidad deseada (`TURN_RESPONSE`).
- **Gravedad**: el Sol y cada planeta atraen la nave dentro de 5 veces su radio. La fuerza baja con el cuadrado de la distancia y se desvanece en el borde del alcance.
- **Colisión**: si la nave queda más cerca de la superficie que `radio + margen`, se la empuja de vuelta y se le quita la velocidad que iba hacia adentro. Por eso "rebota".
- **Control local**: los planetas orbitan, así que se alejarían de una nave quieta. Dentro de 5 radios de un planeta, cada cuadro se le suma a la nave **cuánto se movió ese planeta**. Para quien juega, el planeta queda quieto y solo rota, y se puede rodear con calma.

6. Mapas horneados y mapeo de normales (`normal_map.rs`)

Al arrancar se genera, para cada planeta, una "imagen" con tres capas calculadas con ruido: **pendientes del relieve**, **altura** y un **patrón extra** (nubes, dunas o celdas de Voronoi). Después los shaders solo **consultan** esa imagen, lo que es muy barato por píxel.

Mapeo de normales: en vez de usar la normal del triángulo tal cual, el shader la inclina según las pendientes del mapa. Cambia la luz píxel por píxel y da sensación de relieve fino sin añadir triángulos (`lit_intensity` en `shaders.rs`).

7. Shaders y estilo (`shaders.rs → fragment_shader`)

Cada planeta usa una **técnica distinta**: biomas por altura y nubes (Terran), estratos con ruido ridged (Desierto), celdas de Voronoi con grietas (Hielo), bandas y tormenta analíticas (Gas), corteza con grietas emisivas (Lava).

Después se aplica el **estilo**:

- **Cel shading** (`stylize`): la luz se cuantiza en 3 o 4 escalones, como en un cómic.

- **Semitono**: en el escalón más oscuro se dibujan puntos en una rejilla a 45° (efecto Spider-Verse).
- **Halo de borde** (*rim*): brilla el contorno del planeta, más del lado iluminado.

8. Modelos de superficie (`surface.rs`)

- Hay 4 tipos (árbol, roca, cristal, espiga volcánica) construidos con código a partir de conos, cilindros y una icosfera deformada.
- **Distribución**: se generan direcciones aleatorias con un generador de números con **semilla fija** (`PROP_SEED`), así siempre salen en el mismo lugar. Solo se aceptan donde el **ruido** supera un umbral (así se forman grupos, como bosques) y si no se solapan con otro.
- **Orientación**: cada modelo apunta con su "arriba" en la normal de la superficie.
- **Rotación con el planeta**: cada cuadro se rota la posición y la orientación con la misma rotación del planeta (`rotate_forward`).

9. Rendimiento: descarte y LOD (con el HUD)

- **Descarte (tecla C)**: antes de dibujar un cuerpo se comprueba si su esfera toca el *frustum* (la pirámide visible de la cámara). Si no, se omite. Con los modelos de superficie también se descartan los que están **detrás del horizonte** del planeta.
- **LOD (tecla L)**: según el tamaño aparente del planeta se usa la malla de 320, 1280 o 5120 triángulos. Con L apagado, todos usan la de 5120. La diferencia visual es mínima; el efecto se ve en el contador de triángulos y en los FPS.
- El HUD muestra cuerpos y modelos dibujados/descartados y los triángulos procesados.

9b. Escenario: colapso del universo (tecla K)

Es una máquina de fases (`ScenarioPhase` en `main.rs`):

1. **Colapso (22 s)**: un número `collapse` va de 0 a 1. La posición de cada planeta usa ese número: el radio de su órbita se encoge y el planeta gira más rápido (como un patinador que cierra los brazos). Cuando toca al Sol, "se consume": desaparece y suelta chispas. La gravedad del Sol también aumenta y arrastra a la nave.
2. **Explosión (5 s)**: el Sol se dibuja como una esfera blanca que crece (la onda expansiva), hay 200 chispas y un destello blanco a pantalla completa.
3. **Renacer (3 s)**: todo vuelve a su estado inicial, la nave reaparece en el punto de partida y el blanco se desvanece.

El "calor" del Sol es un parámetro del shader: de 0 (naranja normal) a 1 (blanco). Las chispas son icosaedros pequeños de 20 caras con ese mismo shader.

10. Preguntas probables y respuestas cortas

- **¿Qué es el halo de color del borde de los planetas?** Un efecto de estilo, no una atmósfera. En el shader se compara la normal de la superficie con la dirección a la cámara: donde es casi perpendicular (el contorno) se suma un color. No hay física de atmósfera.

- **¿Qué hace Raylib y qué hace su código?** Raylib: ventana, entrada, mostrar la imagen y texto 2D. Todo lo demás (matrices, rasterización, profundidad, luz, shaders) es del proyecto.

- **¿Por qué no se ve un objeto lejano sobre uno cercano?** Por el depth buffer: cada píxel guarda la profundidad del triángulo más cercano.
- **¿Qué son las coordenadas baricéntricas?** Tres pesos que suman 1 y dicen cuánto cuenta cada vértice en un punto del triángulo; sirven para saber si el punto está dentro e interpolar valores.
- **¿Por qué el movimiento no depende de los FPS?** Porque todo se multiplica por `delta_time`.
- **¿Qué es un fragment shader?** La función que calcula el color de cada píxel de un triángulo. Aquí está en `fragment_shader`.
- **¿Cómo se deforma el planeta?** Cada vértice se mueve a lo largo de su dirección radial una cantidad dada por ruido fractal.
- **¿Cómo sabes que la distribución es reproducible?** Porque usa un generador con semilla fija: la misma semilla da las mismas posiciones.

11. Cosas que te pueden pedir cambiar en vivo

Pedido	Dónde	Qué esperar
Radio de un planeta	`radius` en `PLANETS` (`main.rs`)	Planeta más grande o chico; la malla y los modelos se adaptan
Velocidad orbital	`orbit_speed`	El planeta orbita más rápido o lento
Rotación del planeta	`spin_speed`	Gira más rápido o lento; sus modelos giran con él
Semilla de la distribución	`PROP_SEED`	Otra distribución de árboles, rocas y cristales
Cantidad de modelos	`count` en la lista de props	Más o menos instancias
Relieve del planeta	`terrain_amp`	Montañas más o menos altas
Intensidad del mapa de normales	`normal_strength`	Relieve fino más o menos marcado
Escalones de luz	`levels` en `style`	Más o menos bandas de sombra
Fuerza del semitono	`halftone`	Puntos más o menos visibles
Fuerza de la gravedad	`PLANET_SURFACE_GRAVITY`	La nave se siente atraída más o menos

Después de cambiar un valor, se ejecuta de nuevo con `cargo run --release`.